JOUR 1

Du compose au code

Lundi 19 octobre 2026 - 9h10-17h10 - Infrastructure, orchestration et automatisation

Mastère Data Engineering & IA, M2 - EFREI Bordeaux - Alexandre STOPPINI

AU PROGRAMME

- Relever tout ce qui fait tourner VéloCité v2 : services, dépendances, configuration, secrets
- Transformer le Dockerfile qu'on reçoit souvent en une image légère, non-root, pinnée et scannée
- Faire refuser par la CI une image qui porte une vulnérabilité fixable

JOUR 1

Plan de la journée

390 minutes utiles - deux modules - deux ateliers - fin sur un atelier

Créneau	Bloc	Contenu
09h10 - 09h25	Accueil	Règles du jeu, évaluation
09h25 - 09h40	QCM d'entrée	15 min, ne compte pas dans la note
09h40 - 10h50	Module 1	Du compose au code : les cinq notions
10h50 - 11h05	Pause	
11h05 - 12h30	Atelier 1	L'inventaire de VéloCité v2, en solo
12h30 - 12h40	Point de midi	Qui n'a pas VéloCité qui répond ?
12h40 - 13h40	Déjeuner	
13h40 - 13h55	Binômes	Le dépôt GitHub du binôme, fusion des inventaires
13h55 - 15h00	Module 2	Des images qui se reconstruisent à l'identique
15h00 - 15h15	Pause	
15h15 - 16h55	Atelier 2	Image durcie du job batch, puis la CI
16h55 - 17h10	Bilan	La preuve du jour, ce qui arrive demain

JOUR 1

Règles du jeu

Ce qui vaut pour les six journées

Pratique

- Binôme ou solo : vous choisissez, d'ici 13h40
- Un dépôt GitHub privé par binôme, intervenant invité
- Aucun travail à la maison : tout se fait en séance
- Pauses non négociables, à l'heure dite
- Teams ou LinkedIn : je réponds dans l'heure sur des horaires normaux
- Les démonstrations du 30 novembre sont enregistrées

IAG

Bien sûr l'utilisation de l'IAG ne sera pas encouragée. Allez sur StackOverflow pour chercher vos réponses, ou mieux, demandez-moi.

Sur les ateliers et le dépôt final, toute utilisation de l'IAG non autorisée vaudra une pénalité et des questions très salées et précises lors de la démonstration.

JOUR 1

Comment vous êtes notés

Deux notes, conformes aux trois livrables du syllabus

Dépôt et documentation - 60 %

- Collective, lue dans votre dépôt GitHub
- Reproductibilité : la plateforme renaît depuis zéro en une commande (5 pts)
- Infrastructure décrite en code, images, orchestration (8 pts)
- Workflow et observabilité (4 pts)
- Documentation qu'un autre binôme sait suivre (3 pts)

Démonstration - 40 %

- Individuelle, le 30 novembre, au clavier
- Conduire le scénario : savoir ce qu'on attend avant de taper (6 pts)
- Lire la preuve à l'écran (5 pts)
- Expliquer à un non-spécialiste ce que le geste protège (4 pts)
- Répondre aux questions (5 pts)

JOUR 1

QCM d'entrée

15 minutes - ne compte pas dans votre note

Pourquoi ce QCM

- Me dire d'où part la promotion : Docker, Compose, Git, YAML
- Démonter deux ou trois idées reçues avant le cours
- Vous montrer, réponse par réponse, ce que vous savez déjà
- Seul, sans document : il ne sert à rien autrement

Mode d'emploi

- Lien sur le canal Teams du module
- Une seule bonne réponse par question
- Après l'envoi : cliquez sur "Afficher le score"
- La justification de chaque réponse s'affiche à ce moment

MODULE 1

Du compose au code

- Les cinq notions : service, dépendance, environnement, secret, configuration
- Pourquoi automatiser : le drift, et l'idempotence pour s'en protéger
- Ce que Compose ne sait pas faire, et la pile qui le fera cette semaine

MODULE 1

Lundi matin, 9h12

L'équipe d'exploitation reçoit VéloCité v2 : elle tourne sur un poste

11	35	3 + 3	2
services dans compose.yaml, jobs compris	fichiers de secrets générés par un script	réseaux et volumes à recréer à l'identique	ports publiés, 8443 et 8080, sur 127.0.0.1 seulement

La question qu'on se pose

Le poste de la personne qui a monté VéloCité tombe en panne ce matin. Combien de temps pour la remonter ailleurs, à l'identique, sans elle ?

Et comment savez-vous que c'est "à l'identique" ?

MODULE 1

Les cinq notions du syllabus

Chacune avec son exemple dans VéloCité v2

Notion	Ce que c'est	Dans VéloCité v2
Service	Un programme qui tourne en continu et répond à d'autres	api, ingest, nginx, postgres…
Dépendance	Ce dont un service a besoin pour démarrer ou servir	api attend postgres, redis et seed
Environnement	Une copie complète et isolée de la plateforme	local aujourd'hui ; dev et prod demain
Configuration	Un réglage qui change le comportement, sans être secret	VELOCITE_ENV=local, S3_ENDPOINT
Secret	Une valeur qui donne un accès si elle fuit	jwt_secret, pg_svc_api, nginx.key
Job	Un programme qui s'exécute, finit, et rend un code de sortie	seed, init-stockage, batch

MODULE 1

Pourquoi automatiser : le drift

Ce qui se fait à la main s'écarte de ce qui est écrit, sans bruit

Le mécanisme

- Drift : l'écart entre ce qui tourne et ce qui est décrit
- Chaque geste manuel le crée : une commande, un fichier copié, un correctif urgent
- Il ne se voit pas tant que rien ne casse
- Il se paie le jour où il faut recréer : migration, panne, deuxième environnement
- Question : sur votre poste, que reste-t-il de septembre ?

Rejoué le 16 octobre, en préparant ce cours

```
PS> docker compose ls
NAME       STATUS       CONFIG FILES
velocite   running(7)   [...]/socle/app/compose.yaml
PS> docker volume ls --format "{{.Name}}"
[...]
velocite_garagedata
velocite_pgdata
velocite_redisdata
PS> docker compose up -d
[...] Bind for 127.0.0.1:8080 failed: port is already allocated
PS> docker compose ps -a --format "table {{.Service}}\t{{.Status}}"
[...]
loadgen         Created
nginx           Created
[...]
# Un ancien projet "velocite", oublié, tenait le port 8080.
# Rien dans compose.yaml ne pouvait le dire.
```

MODULE 1

Le remède : décrire le desired state, pas les gestes

Le même geste rejoué deux fois, sous deux formes : un ordre, une description

PowerShell - un ordre, rejoué

```
PS> docker run -d --name essai-redis `
>>    mirror.gcr.io/library/redis:8.8.3-alpine
632b457f8b0cd1efbffef1b87578c043679be2[...]
PS> docker run -d --name essai-redis `
>>    mirror.gcr.io/library/redis:8.8.3-alpine
docker: Error response from daemon: Conflict.
The container name "/essai-redis" is already
in use by container "632b457f8b0c[...]". [...]
PS> $LASTEXITCODE
125
```

PowerShell - une description, rejouée

```
PS> docker compose up -d        # second passage
 Container velocite-v2-loadgen-1 Running
 Container velocite-v2-postgres-1 Running
 Container velocite-v2-redis-1 Running
 Container velocite-v2-ingest-1 Running
 Container velocite-v2-garage-1 Running
 Container velocite-v2-api-1 Running
 Container velocite-v2-nginx-1 Running
[...]
 Container velocite-v2-init-stockage-1 Started
[...]
 Container velocite-v2-seed-1 Started
[...]
PS> $LASTEXITCODE
0
```

À gauche, un ordre : rejoué, il échoue. À droite, une description : Compose compare compose.yaml à ce qui tourne et ne fait que la différence. Rejouer ne change rien, c'est l'idempotence.

Question : pourquoi seed et init-stockage repartent-ils, et pourquoi n'est-ce pas grave ?

MODULE 1

Les dépendances de VéloCité

La description dit aussi qui attend qui : depends_on et une condition

PowerShell - qui attend qui, lu dans la configuration

```
PS> $c = docker compose config --format json | ConvertFrom-Json
PS> foreach ($s in $c.services.PSObject.Properties) {
>>   foreach ($d in $s.Value.depends_on.PSObject.Properties) {
>>     "{0,-13} attend {1,-13} {2}" -f $s.Name, $d.Name, $d.Value.condition } }
api           attend postgres      service_healthy
api           attend redis         service_healthy
api           attend seed          service_completed_successfully
ingest        attend postgres      service_healthy
init-stockage attend garage        service_started
loadgen       attend nginx         service_healthy
nginx         attend api           service_healthy
nginx         attend ingest        service_healthy
seed          attend init-stockage service_completed_successfully
seed          attend postgres      service_healthy
# service_started : lancé, sans plus - service_healthy : son
# healthcheck répond - service_completed_successfully : job fini en 0
```

Ce qui casse demain

- Kubernetes n'a pas d'ordre de démarrage
- Chaque service doit savoir attendre ses dépendances
- Ou se déclarer "pas prêt" tant qu'elles manquent : /readyz
- Question : PostgreSQL redémarre. Que devient l'API ?

MODULE 1

Configuration et secrets

Ce que chaque service reçoit, et le chemin d'un secret jusqu'au programme

Le chemin d'un secret

- 1. init-secrets écrit 35 fichiers dans app/secrets, ignoré par Git
- 2. compose.yaml en déclare 34 (bloc secrets:) et dit qui reçoit lequel
- 3. Docker les monte en fichiers dans /run/secrets du conteneur
- 4. Le programme lit NOM_FILE et ouvre le fichier. Absent : exit 78
- La configuration, elle, passe en clair : VELOCITE_ENV=local
- Question : 35 générés, 34 déclarés. Lequel manque, et pourquoi ?

compose.yaml, lignes 146 à 156 - api

```
environment:
  VELOCITE_ENV: local
  DATABASE_URL_FILE: /run/secrets/database_url_api
  REDIS_URL_FILE: /run/secrets/redis_url_api
  S3_ENDPOINT: http://garage:3900
  S3_ACCESS_KEY_FILE: /run/secrets/s3_api_key_id
  S3_SECRET_KEY_FILE: /run/secrets/s3_api_secret
  JWT_SECRET_FILE: /run/secrets/jwt_secret
  PSEUDO_KEY_FILE: /run/secrets/pseudo_key
secrets: [database_url_api, redis_url_api,
  s3_api_key_id, s3_api_secret, jwt_secret,
  pseudo_key, ca.crt]

# 78 : sys.exit(78) dans services/api/config.py
# app/README.md, Codes de sortie : "configuration
# absente ou invalide", ne pas relancer
```

MODULE 1

Ce qui change d'un environnement à l'autre

Et tout ce qui doit rester identique

Doit changer

- La valeur de chaque secret : jamais partagée entre dev et prod
- Le nom de l'environnement : VELOCITE_ENV
- Le port publié de nginx : 8443 en dev, 9443 en prod
- Le nombre de replicas des services sans état
- La version de l'API : dev d'abord, prod ensuite

Doit rester identique

- Les images, au digest près
- Les noms des services et leurs dépendances
- Le durcissement : non-root, système de fichiers en read-only
- Les règles réseau
- Sinon, ce qui marche en dev ne prouve rien pour la prod

MODULE 1

Ce que Compose ne sait pas faire

Et le jour de la semaine où on y répond

Besoin	Avec Compose	Réponse retenue	Quand
Deux environnements sur un poste	deux projets -p, à la main	un namespace chacun, overlays Kustomize	mar., mer.
Secrets générés, jamais dans le dépôt	fichiers en clair, un jeu par poste	OpenTofu, state chiffré	mardi
Plusieurs replicas derrière un nom	--scale, sans regarder la santé	Deployment et Service	mercredi
Rolling update et rollback	coupure, aucun historique	rollout, rollout undo	mercredi
Job planifié avec retries	aucun planificateur	Argo Workflows	jeudi
Supervision	/metrics exposées, lues par personne	Prometheus, Grafana, Loki	jeudi

MODULE 1

La pile de la semaine

Chaque outil en une ligne, plus ce qu'on a écarté

Les outils

- OpenTofu : décrit l'infrastructure en code, garde le state de ce qu'il a créé
- Kubernetes : orchestrateur, fait tourner et surveille les conteneurs
- kind : un cluster Kubernetes dans des conteneurs Docker, jetable
- Kustomize : une base commune, un overlay par environnement
- Argo Workflows : enchaîne des jobs, relance ceux qui échouent
- Prometheus, Grafana, Loki : métriques, dashboards, logs

Écarté, et pourquoi

- Airflow : 3 à 4 Go de plus sur vos postes ; présenté jeudi, pas installé
- MinIO : dépôt communautaire archivé le 25 avril 2026, remplacé par Garage
- Promtail : fin de vie le 2 mars 2026, remplacé par Alloy
- Docker Hub en direct : limité à 100 pulls par 6 h sans compte ; on tire de mirror.gcr.io

ATELIER 1

Atelier 1 - L'inventaire de VéloCité v2

80 min - en solo - pièces : docs/INVENTAIRE.md et le croquis de docs/ARCHITECTURE.md

Consignes

- Clonez le dépôt du cours (adresse sur Teams) dans C:\cours, ou ~/cours sous macOS et Linux
- Lancez VéloCité depuis le dossier app : les commandes du panneau
- Vérifiez : init-stockage et seed en Exited (0), tous les autres Up
- Ouvrez docs/INVENTAIRE.md : chaque section pose une question et donne sa commande. Lancez-la, remplissez
- Section 4 : 35 fichiers générés, 34 déclarés. Lequel manque, et pourquoi ?
- Croquis : docs/ARCHITECTURE.md génère les flèches ; ajoutez le poste et les réseaux
- Section 7 : une phrase sans terme technique

Windows - PowerShell

```
# Une fois : répondez O
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
New-Item -ItemType Directory -Force C:\cours
cd C:\cours
git clone <adresse du dépôt du cours, sur Teams>
cd velocite-ops/app
./scripts/init-secrets.ps1
docker compose up -d
docker compose ps -a

# macOS, Linux : mkdir -p ~/cours && cd ~/cours,
#   puis ./scripts/init-secrets.sh à la place
#   (fiche Commandes du J1)
# La suite : docs/INVENTAIRE.md
```

ATELIER 1

Ce qu'on attend de l'inventaire

Les critères, pas la réponse

Un bon inventaire

- Chaque chiffre a sa commande : combien de services, de secrets, de réseaux
- Chaque service est classé : service, job, ou ni l'un ni l'autre
- Les secrets sont rangés par famille, avec qui les lit
- Les secrets dérivés sont repérés
- Ce que vous n'avez pas compris est écrit en section 6, pas caché

Légende - sur un exemple neutre

```
| Élément    | Classe  | Lu par       | Source         |
|------------|---------|--------------|----------------|
| mailer     | service | -            | compose.yaml   |
| smtp_pass  | secret  | mailer       | compose.yaml   |
| SMTP_URL   | dérivé  | mailer       | init-secrets   |

# Classe : service / job / configuration /
#          secret / dérivé
# Source : le fichier ou la commande qui le
#          prouve. Sans source, la ligne va
#          en section 6, questions ouvertes.
```

JOUR 1

Point de midi

Dix minutes, une question qui fait lever la main

La question

Qui n'a pas VéloCité qui répond ?

Le test, en une commande, depuis app :

curl.exe --ssl-no-revoke --cacert secrets/ca.crt https://localhost:8443/api/version

(macOS et Linux : curl --cacert secrets/ca.crt ...)

Réponse attendue, rejouée le 16 octobre :

{"environnement":"local",

"service":"velocite-api","version":"2.0.0"}

Votre phrase, relue par vous

Relisez votre phrase de la section 7 comme si vous ne l'aviez pas écrite.

Pour chaque mot, demandez-vous : quelqu'un qui ne fait pas d'informatique le comprendrait-il ?

Si non, remplacez-le avant de partir déjeuner.

ATELIER 1

Votre binôme, votre dépôt

15 min - le dépôt du binôme naît, les deux inventaires de ce matin fusionnent

Dans l'ordre

- Binôme formé ? Sinon, solo : faites la partie A
- A crée sur GitHub un dépôt privé vide et invite B et moi
- Son nom : velocite-ops-NOM1-NOM2
- A, dans son clone de ce matin : la partie A du panneau
- B, dans son clone de ce matin aussi : la partie B. Ses secrets et VéloCité restent en place
- Comparez docs et ../mes-docs ligne par ligne ; un désaccord va en section 6

Windows, macOS, Linux - depuis velocite-ops

```
# A
git remote rename origin cours
git remote add origin <adresse du dépôt>
git ls-files app/secrets          # aucune ligne
git check-ignore -v app/secrets/jwt_secret
git add docs
git commit -m "Inventaire, atelier 1"
git push -u origin main
# B, une fois que A a poussé
git remote rename origin cours
git remote add origin <adresse du dépôt>
cp -r docs ../mes-docs     # garde son inventaire
git fetch origin
git reset --hard origin/main
git branch -u origin/main
# Chaque matin : git pull --no-rebase --no-edit cours j2
```

MODULE 2

Des images qui se reconstruisent à l'identique

- Ce qu'est une image : layers, cache, registry, tag et digest
- Le Dockerfile qu'on reçoit souvent, mesuré, et ses défauts un par un
- Pinner, multi-stage, non-root, arrêt propre, scan, refus en CI

MODULE 2

Image, layer, registry, tag, digest

Six mots pour lire n'importe quel Dockerfile

Mot	Ce que c'est	Exemple
Image	Un système de fichiers figé et une commande de démarrage	`velocite/batch:1.0.0`
Layer	Le résultat d'une instruction du Dockerfile, empilé et mis en cache	`RUN pip install ...`
Registry	Un entrepôt d'images, d'où on les tire (pull)	`mirror.gcr.io, ghcr.io`
Tag	Un nom lisible, qui peut changer de cible	`python:3.13.16-slim-trixie`
Digest	Le sha256 du contenu : une image, une seule	`@sha256:70729b46c69b...`
Dockerfile	La recette, lue de haut en bas ; ENTRYPOINT lance, CMD donne les arguments	`FROM, COPY, RUN, ENTRYPOINT`

MODULE 2

Le Dockerfile qu'on reçoit souvent

app/services/batch/Dockerfile.naif - il marche

app/services/batch/Dockerfile.naif

```
# Le Dockerfile qu'on reçoit souvent. Il marche. Combien de problèmes
# comptez-vous ? (support de cours, module 2 de la journée 1)
FROM python:latest
COPY . .
RUN pip install psycopg[binary] boto3
CMD python batch.py extract

# Deux outils, depuis app/services/batch. Windows :
Get-Content Dockerfile.naif | docker run --rm -i `
  mirror.gcr.io/hadolint/hadolint:v2.14.0 hadolint --no-color -
# macOS et Linux :
docker run --rm -i mirror.gcr.io/hadolint/hadolint:v2.14.0 \
  hadolint --no-color - < Dockerfile.naif
# Partout :
docker build --check -f Dockerfile.naif .
```

À vous

Combien de problèmes dans ces quatre lignes ? Six minutes, en binôme.

Notez-les dans app/services/batch/defauts.txt : la ligne, ce qui peut arriver, et quand.

Trois sources : votre lecture, hadolint, docker build --check.

Ne construisez pas l'image : près de 500 Mo à télécharger, plus de 1,2 Go sur le disque.

Indice : on en a relevé onze ; les outils en trouvent cinq.

MODULE 2

Ce que donnent ces quatre lignes

Mesuré les 13 et 16 octobre 2026 : docker image inspect, id, Trivy 0.70.0

> 1,2 Go	uid=0	337	3.15.0
1,21 ou 1,77 Go selon le stockage d'images de Docker, pour 13 Ko de script	root : le batch peut tout faire dans le conteneur	HIGH ou CRITICAL ; fixables : 4 le 13 octobre, 20 le 16	version de Python que personne n'a choisie

Ce qu'on visera cet après-midi

- Moins de 300 Mo sur le disque - non-root - zéro vulnérabilité HIGH ou CRITICAL fixable
- Une ligne de code changée : seul le dernier layer est refait, pas une réinstallation complète

MODULE 2

Les layers et le cache

Kit de démonstration : une ligne ajoutée à rapport.py, puis un build

Dockerfile.desordre

```
# Extrait du Dockerfile
COPY . .
RUN pip install --no-cache-dir [...] -r requirements.txt

# Une ligne de plus dans rapport.py, puis :
# docker build --progress=plain -f Dockerfile.desordre .
[3/4] COPY . .
DONE 0.3s
[4/4] RUN pip install --no-cache-dir [...]
DONE 10.7s

# Durée totale : 14,7 s. COPY . . a changé :
# tout ce qui suit est refait, pip réinstalle.
# (préfixes #N retirés ; étapes comptées sans la ligne du proxy)
```

Dockerfile.ordre

```
# Extrait du Dockerfile
COPY requirements.txt .
RUN pip install --no-cache-dir [...] -r requirements.txt
COPY rapport.py .

# La même modification, puis le même build :
[3/5] COPY requirements.txt .
CACHED
[4/5] RUN pip install --no-cache-dir [...]
CACHED
[5/5] COPY rapport.py .
DONE 0.1s
# Durée totale : 0,5 s.
# (préfixes #N retirés ; étapes comptées sans la ligne du proxy)
```

MODULE 2

Le multi-stage build

Ce qui ne sert qu'au build reste dans le premier stage

Comment ça marche

- Stage 1, le build : pip construit un venv, puis on retire pip
- Stage 2 : une image neuve ; COPY --from ne prend que /opt/venv
- ENV PATH : le venv copié devient le Python par défaut
- Mesuré sur le kit : 272 Mo en single-stage, 265 Mo en multi-stage. Ici, seul pip part
- Si une dépendance se compile : gcc et libpq-dev, 302 Mo, restent au stage 1
- Question : dans l'image du batch, qu'est-ce qui ne sert qu'au build ?

app/services/loadgen/Dockerfile ([...] = omis)

```
ARG PYTHON_IMAGE=mirror.gcr.io/library/python:3.13.16-sl[...]
FROM ${PYTHON_IMAGE} AS construction
ENV PIP_DISABLE_PIP_VERSION_CHECK=1 PIP_NO_CACHE_DIR=1
RUN python -m venv /opt/venv
COPY requirements.txt /tmp/requirements.txt
RUN /opt/venv/bin/pip install --require-hashes [...] \
 && /opt/venv/bin/pip uninstall -y -q pip
FROM ${PYTHON_IMAGE}
[...]
ENV PATH=/opt/venv/bin:$PATH [...]
RUN python -m pip uninstall -y -q pip \
 && useradd --system --uid 10001 [...] velocite
COPY --from=construction /opt/venv /opt/venv
WORKDIR /app
COPY *.py ./
USER 10001:10001
[...]
CMD ["python", "loadgen.py"]
```

MODULE 2

Pinner ce qu'on tire

L'image de base, les dépendances, et le registry lui-même

Trois pins

- Image de base : un tag précis ET son digest, FROM nom:tag@sha256:…
- Dépendances Python : versions et hashes dans requirements.txt, pip --require-hashes
- Registry : mirror.gcr.io, mêmes digests que Docker Hub, sans sa limite de pulls
- Question : quelqu'un remplace boto3 1.43.111 sur le serveur de paquets. Que fait pip ?

Sans pin, puis le digest d'un tag

```
# Dockerfile.naif, build le 12 puis le 16 octobre
PS> docker run --rm --entrypoint python `
>>    velocite/batch:naif-12oct --version
Python 3.14.8
PS> docker run --rm --entrypoint python `
>>    velocite/batch:naif --version
Python 3.15.0
# boto3 : 1.43.111 le 12, 1.43.114 le 16

# Le digest d'un tag, sans rien télécharger :
PS> docker buildx imagetools inspect `
>>    mirror.gcr.io/library/python:3.13.16-slim-trixie
Name:      mirror.gcr.io/library/python:3.13.16-sl[...]
MediaType: application/vnd.oci.image.index.v1+json
Digest:    sha256:70729b46c69b4f1e97c4822c1af3df[...]
# macOS, Linux : \ en fin de ligne au lieu de `
```

MODULE 2

Non-root et read-only

Ce que le conteneur a le droit de faire une fois lancé

Deux réglages

- USER 10001:10001 : le programme tourne avec un compte sans droit
- --read-only : le système de fichiers de l'image ne s'écrit plus
- PYTHONDONTWRITEBYTECODE=1 : Python n'essaie plus d'écrire ses caches
- Question : que gagne un attaquant qui prend la main sur un programme root ?

PowerShell - qui suis-je, puis read-only

```
PS> docker run --rm --entrypoint id velocite/batch:naif
uid=0(root) gid=0(root) groups=0(root)
PS> docker run --rm --entrypoint id demo/rapport:deux-etapes
uid=10001 gid=10001 groups=10001

PS> docker run --rm --read-only --entrypoint python `
>>   velocite/batch:naif -c "open('/essai', 'w')"
[...]
OSError: [Errno 30] Read-only file system: '/essai'

PS> docker run --rm --read-only demo/rapport:deux-etapes
rapport : Python 3.13.16, boto3 1.43.111, psycopg 3.3.6

# Une image bien faite tourne en read-only ;
# rien ne peut y déposer un fichier.
```

MODULE 2

Arrêter un conteneur proprement

Ce que fait docker stop, et pourquoi un programme peut l'ignorer

Ce que fait docker stop

- 1. Il envoie SIGTERM au PID 1, le premier programme du conteneur
- 2. Il attend 10 secondes
- 3. Il envoie SIGKILL : tué net, code 137
- Le piège : un signal que le PID 1 ne traite pas est ignoré
- Deux remèdes : --init, qui reçoit et transmet ; ou un handler
- Question : quelle étape du bilan supporte le moins d'être coupée ?

Kit : quatre conteneurs, puis docker stop

```
# shell form : sh -c "python rapport.py --attendre 600"
# exec form  : python rapport.py --attendre 600
PS> docker top forme-shell -o pid,args
PID       COMMAND
18329     sh -c python rapport.py --attendre 600
18343     python rapport.py --attendre 600
PS> foreach ($n in "forme-shell", "forme-exec",
>>   "avec-init", "arret-propre") { [...] }
forme-shell   10.2 s  code 137
forme-exec    10.2 s  code 137
avec-init     0.2 s  code 143
arret-propre  0.2 s  code 0
# 137 : tué au bout du délai
# 143 : arrêté par SIGTERM
# 0   : le programme a fini lui-même
```

MODULE 2

Scanner une image avec Trivy

Prouver ce que l'image contient : bloquer ce qui est fixable, lister le reste

La politique du module

- CVE : identifiant public d'une vulnérabilité connue, avec l'année et un numéro
- Severity : LOW, MEDIUM, HIGH, CRITICAL, tirée d'une note sur 10
- Fixable : une version publiée corrige la vulnérabilité
- Bloquant : HIGH ou CRITICAL et fixable. Le reste : listé, surveillé
- Image naïve, 16 octobre : 337 HIGH ou CRITICAL, dont 20 fixables. Refusée

PowerShell - sans monter le socket Docker

```
PS> docker save velocite/batch:naif -o batch.tar
PS> docker run --rm -v "${PWD}:/w" `
>>   -v trivy-cache:/root/.cache `
>>   mirror.gcr.io/aquasec/trivy:0.70.0 `
>>   image --input /w/batch.tar `
>>   --severity HIGH,CRITICAL `
>>   --ignore-unfixed --exit-code 1 --quiet
[...]
Total: 16 (HIGH: 16, CRITICAL: 0)      # Debian
[...]
Total: 4 (HIGH: 4, CRITICAL: 0)        # Python
PS> $LASTEXITCODE
1
# On scanne une archive : le conteneur
# de scan n'a aucun accès au moteur.
```

MODULE 2

Faire refuser l'image par la CI

GitHub Actions : des steps qui échouent au bon moment, des actions pinnées par SHA

Ce qu'il faut savoir

- À chaque push sur main et à chaque pull request, GitHub lance une machine Linux et joue les steps
- Un step qui rend un code différent de 0 arrête le job : il devient rouge
- uses: appelle une action, du code écrit par d'autres, qui voit vos secrets
- Un tag d'action peut être déplacé ; un SHA de commit, jamais
- 19 mars 2026 : 76 tags de trivy-action sur 77 déplacés vers un code voleur de secrets
- Les workflows pinnés par SHA ont continué d'exécuter l'ancien code, sain

.github/workflows/ci.yml - fourni

```
on:
  push:
    branches: [main]
  pull_request:
[...]
jobs:
  images:
    [...]
    steps:
      # Pinnée par SHA : ce commit, pour toujours
      - uses: actions/checkout@3d3c42e5[...] # v7.0.1
        [...]
      # Un step : un nom, une commande
      - name: Outils
        run: bash scripts/ci/outils.sh trivy
      # A FAIRE : construction, analyse, utilisateur.
```

ATELIER 2

Atelier 2 - L'image durcie du job batch

Partie 1, 55 min - en binôme - pièce : app/services/batch/Dockerfile

Consignes

- Créez app/services/batch/Dockerfile en partant de app/services/loadgen/Dockerfile, déjà durci
- Image de base : python, tag précis en slim, avec son digest (slide 26), depuis mirror.gcr.io
- Multi-stage : le venv se construit dans le premier stage (slide 25)
- Dépendances : le requirements.txt fourni, avec --require-hashes
- Ordre des layers : une ligne de batch.py changée, seule la copie du code est refaite
- USER 10001:10001 et ENTRYPOINT ["python", "/app/batch.py"] en exec form : extract ou --help s'ajoutent derrière
- Les vérifications du panneau passent ; chaque ligne commentée par le défaut de defauts.txt qu'elle corrige

PowerShell - les vérifications, depuis la racine

```
docker build -t velocite/batch:1.0.0 app/services/batch
docker image inspect velocite/batch:1.0.0 `
  --format "{{.Size}} {{.Config.User}}"
# -> une taille sous 300000000 octets, puis 10001:10001
docker run --rm --read-only velocite/batch:1.0.0 --help
# -> l'aide de batch.py s'affiche
docker save velocite/batch:1.0.0 -o batch.tar
# puis Trivy, slide 29 : $LASTEXITCODE = 0
cd app
docker compose --profile batch run --rm `
  batch extract --date 2026-03-12
# -> event=extract_ok date=2026-03-12 lignes=64
# macOS, Linux : fiche Commandes du J1
```

ATELIER 2

Atelier 2 - La CI refuse une image dangereuse

Partie 2, 40 min - en binôme - pièce : le job images de .github/workflows/ci.yml

Consignes

- Complétez le job images : trois steps après "Outils", sur son modèle
- Build : docker build, image taguée velocite/batch:ci
- Scan : trivy image, HIGH et CRITICAL, --ignore-unfixed, --exit-code 1
- Utilisateur : docker image inspect, le step échoue si ce n'est pas 10001:10001
- Poussez, ouvrez l'onglet Actions : le job doit être vert
- Prouvez qu'il sait refuser : une branche qui build Dockerfile.naif, une pull request non fusionnée, un job rouge
- Une phrase sans terme technique : ce que ce contrôle évite

Les commandes de la CI tournent sous Linux

```
# Dans un step "run:", syntaxe bash :
docker build -t velocite/batch:ci \
  app/services/batch

trivy image --severity HIGH,CRITICAL \
  --ignore-unfixed --exit-code 1 \
  velocite/batch:ci

u=$(docker image inspect \
  --format '{{.Config.User}}' velocite/batch:ci)
test "$u" = "10001:10001"

# Un step échoue dès qu'une commande
# rend un code différent de 0.
```

ATELIER 2

Ce qu'on attend de l'atelier 2

Les preuves à garder, pas la solution

Dans votre dépôt ce soir

- app/services/batch/Dockerfile, chaque ligne commentée : le défaut qu'elle corrige
- Le job images complet dans .github/workflows/ci.yml
- Un job vert sur main, un job rouge sur la branche de l'image naïve
- Les deux liens, et vos mesures (taille, utilisateur, Trivy), dans le message du dernier commit

Les questions du 30 novembre

Pour chaque ligne de votre Dockerfile, je pourrai demander : qu'est-ce qui se passe si on la retire ?

Pour la CI : montrez-moi qu'elle sait refuser.

Préparez la réponse maintenant, pendant que vous savez pourquoi vous avez écrit chaque ligne.

BILAN

Bilan de la journée 1

CE QUI EST DANS VOTRE DÉPÔT

- L'inventaire de VéloCité v2, fusionné à deux
- Le croquis d'architecture
- Le Dockerfile du batch : moins de 300 Mo, non-root
- Une CI qui refuse l'image naïve, et la preuve qu'elle refuse
- Deux phrases sans terme technique

DEMAIN : L'INFRASTRUCTURE DÉCRITE EN CODE

- OpenTofu crée le cluster kind, puis l'environnement dev
- Avant de partir, depuis app : docker compose down
- Puis : docker info --format "{{.CgroupVersion}}" doit répondre 2
- Si c'est 1 : venez me voir demain à 9h05
- En arrivant : git pull --no-rebase --no-edit cours j2
- Puis git push
- Rien à faire ce soir (promis ce n'est pas une ruse)

ALEXANDRE STOPPINI - Teams ou LinkedIn - je réponds dans l'heure sur des horaires normaux